

CSE 4554: Machine Learning

Afrin Jahan Era

ID: 220042132

Department: CSE

Programme: SWE

ML_Lab_Task03_Report

TASK 1: Logistic Regression from Scratch (Binary)

TODO 1: Sigmoid Function

The sigmoid function is used to map the linear output of the model into a probability value between 0 and 1. In logistic regression, this function allows us to interpret the model's output as the probability that a given input belongs to the positive class.

Mathematically, the sigmoid function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This function is applied element-wise to the linear combination $z = X\theta$. Numerical clipping is used to prevent overflow during exponential calculation, which does not alter the mathematical formula.

Explanation of todos:

1. Converts linear output into probability
2. Output range: (0, 1)
3. Used in hypothesis function

```
def sigmoid(z):  
    z = np.clip(z, -500, 500) # numerical stability  
    return 1 / (1 + np.exp(-z))
```

TODO 2: Cost Function (No Regularization)

The cost function measures how well the logistic regression model predicts the true class labels. For binary classification, the binary cross-entropy loss is used.

The cost function penalizes confident but wrong predictions more heavily than uncertain ones. The average loss over all training examples is computed as:

Formula (Binary Cross Entropy)

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

This loss function encourages predicted probabilities to be close to the true labels.

Explanation of the todo:

1. Measures prediction error
2. Lower cost $\rightarrow$ better model

```
def compute_cost_no_reg(X, y, theta):  
    m = len(y)  
    h = sigmoid(X @ theta)  
    eps = 1e-15  
    return -(1/m) * (y @ np.log(h + eps) + (1-y) @ np.log(1-h + eps))
```

TODO 3: Cost Function with Regularization

Regularization is added to the cost function to prevent overfitting by discouraging excessively large model parameters.

L2 Regularization (Ridge)

L2 regularization adds the squared magnitude of the weights to the cost function:

$$J(\theta) = J_{no_reg} + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

This encourages smaller weights and smoother decision boundaries.

L1 Regularization (Lasso)

L1 regularization adds the absolute value of the weights:

$$J(\theta) = J_{no_reg} + \frac{\lambda}{m} \sum_{j=1}^n |\theta_j|$$

This can force some weights to become exactly zero, leading to sparse models. In both cases, the bias term θ_0 is excluded from regularization, as stated in the slides.

```
def compute_cost(X, y, theta, lambda_=0.0, reg_type=None):
    m = len(y)
    cost = compute_cost_no_reg(X, y, theta)

    if lambda_ == 0 or reg_type is None:
        return cost

    theta_no_bias = theta[1:]

    if reg_type.lower() == 'l2':
        reg = (lambda_ / (2*m)) * np.sum(theta_no_bias**2)
    elif reg_type.lower() == 'l1':
        reg = (lambda_ / m) * np.sum(np.abs(theta_no_bias))
    else:
        raise ValueError("reg_type must be 'l1', 'l2', or None")

    return cost + reg
```

TODO 4: Gradient Computation

The gradient represents the direction and magnitude by which model parameters should be updated to minimize the cost function. The gradient of the regularised cost function is:

Base Gradient Formula

$$\nabla J(\theta) = \frac{1}{m} X^T (h - y)$$

This expression calculates how much each parameter contributes to the prediction error.

Gradient with Regularization

1. L2 Regularization adds a term proportional to

L2 Gradient

$$+ \frac{\lambda}{m} \theta_j$$

2. L1 Regularization adds a term proportional to

L1 Gradient (Subgradient)

$$+ \frac{\lambda}{m} \text{sign}(\theta_j)$$

These additional terms penalize large weights during optimization.

```
def compute_gradient(X, y, theta, lambda_=0.0, reg_type=None):
    m = len(y)
    h = sigmoid(X @ theta)
    grad = (1/m) * (X.T @ (h - y))

    if lambda_ != 0 and reg_type is not None:
        if reg_type.lower() == 'l2':
            grad[1:] += (lambda_/m) * theta[1:]
        elif reg_type.lower() == 'l1':
            grad[1:] += (lambda_/m) * np.sign(theta[1:])
        else:
            raise ValueError("reg_type must be 'l1', 'l2', or None")

    return grad
```

TODO 5: Gradient Descent with Loss Tracking

Gradient descent is an iterative optimization algorithm used to minimize the cost function by updating parameters in the opposite direction of the gradient.

The update rule is:

$$\theta := \theta - \alpha \nabla J(\theta)$$

Where:

1. α is the learning rate
2. $\nabla J(\theta)$ is the gradient of the cost function

During training, both training loss and validation loss are recorded at each iteration to monitor learning progress and detect overfitting.

Explanation of todo:

1. Repeatedly updates parameters
2. Tracks train & validation loss

```
def gradient_descent_loss_tracking(  
    X_train, y_train, X_val, y_val,  
    theta, alpha, num_iters,  
    lambda_=0.0, reg_type=None  
):  
    train_losses = []  
    val_losses = []  
  
    for i in range(num_iters):  
        grad = compute_gradient(X_train, y_train, theta, lambda_, reg_type)  
        theta = theta - alpha * grad  
  
        train_losses.append(compute_cost(X_train, y_train, theta, lambda_, reg_type))  
        val_losses.append(compute_cost(X_val, y_val, theta, lambda_, reg_type))  
  
        if i % 200 == 0:  
            print(f'Iter {i}: Train Loss={train_losses[-1]:.6f}, Val Loss={val_losses[-1]:.6f}')  
  
    return theta, train_losses, val_losses
```

TASK 2: One-vs-All Multiclass Logistic Regression

Since logistic regression is inherently a binary classifier, the One-vs-All (OvA) strategy is used for multiclass classification.

In this approach:

1. A separate binary classifier is trained for each class
2. Each classifier predicts the probability that a sample belongs to its class

For each class k :

$$h_k(x) = \sigma(X\theta_k)$$

Final prediction:

$$\hat{y} = \arg \max_k h_k(x)$$

The class with the highest predicted probability is selected as the final output.

```
def predict_ova_mc(X_bias, theta_matrix):  
    probs = sigmoid(X_bias @ theta_matrix.T)  
    return np.argmax(probs, axis=1)
```

TASK 3: Scikit-Learn Models

TASK 3.1: Decision Tree (Given)

The Decision Tree classifier splits the data into regions based on feature values to minimize class impurity. The model is trained on the training dataset and evaluated on the test dataset using accuracy and a confusion matrix.

TASK 3.2: KNN

The KNN algorithm classifies a data point based on the majority class among its k nearest neighbors in the feature space. After training (which involves storing the dataset), predictions are made for test samples and evaluated using accuracy and a confusion matrix.

```
knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)
pred_knn = knn.predict(X_test)

print("KNN Accuracy:", accuracy_score(y_test, pred_knn))
cm = confusion_matrix(y_test, pred_knn)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=target_names, yticklabels=target_names)
plt.title("KNN Confusion Matrix")
plt.show()
```

TASK 3.3: Naive Bayes

The Naive Bayes classifier is based on Bayes' Theorem and assumes conditional independence between features. The Gaussian Naive Bayes variant is used for continuous features. The model is trained on the training data and evaluated on the test data.

```
nb = GaussianNB()

nb.fit(X_train, y_train)
pred_nb = nb.predict(X_test)

print("Naive Bayes Accuracy:", accuracy_score(y_test, pred_nb))
cm = confusion_matrix(y_test, pred_nb)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=target_names, yticklabels=target_names)
plt.title("Naive Bayes Confusion Matrix")
plt.show()
```